



Département de génie informatique et génie logiciel

INF2010

Structures de données et algorithmes

Rapport TP2

Soumis par

Nicolas Mihali (2435049)

Akhan Mehmet Sozen (2420324)

Groupe 2

15 février 2026

Polytechnique Montréal

1. Décrivez la logique de votre solution optimale en temps et analysez la complexité temporelle et spatiale.

Logique de la solution:

La solution vise à trouver une paire d'éléments dans une liste `nums` dont la somme est égale à `k`. Elle commence par créer et copier la liste `nums` dans une nouvelle liste `sortedNums` et trie dans l'ordre croissant les éléments contenus dans la liste.

L'index `bi` (b index) commence à 0 et `ai` (a index) commence à la fin de la liste (`sortedNums.size() - 1`). Tant que `bi` est inférieur à `ai`, la solution continue à chercher une paire. On commence par calculer la somme (`sum`) des indexes initiaux, lorsque la somme est égale à `k` on retourne une paire, si la somme est inférieure à `k` alors on incrémente `bi` sinon on décrémente `ai`. Si aucune paire n'est trouvée, la fonction retourne `null`.

Complexité temporelle

- `List<Integer> sortedNums = new ArrayList<>(nums);` se fait en $O(n)$
- `Collections.sort(sortedNums);` se fait en $O(n \log(n))$
- L'initialisation de `ai` et `bi` se fait en $O(1)$
- La boucle s'exécute n fois car `bi` et `ai` se déplacent vers le centre, chaque index étant visité au maximum une fois
 - `sortedNums.get(ai)` et `sortedNums.get(bi)` se fait en $O(1)$
 - `sortedNums.get(ai) + sortedNums.get(bi)` se fait en $O(1)$
 - `sum` se fait en $O(1)$ soit $O(1+1+1+1) = O(4)$ pour la ligne de `sum`
 - `if(sum == k)` se fait en $O(1)$
 - `else if(sum < k)` se fait en $O(1)$
 - `bi++` se fait en $O(1)$
 - `ai--` se fait en $O(1)$
 - la boucle est donc $O(n * (4+1+1+1+1)) = O(n)$
- La fonction s'exécute en $O(n + n \log(n) + n) = O(n \log(n))$

Complexité spatiale

- `List<Integer> sortedNums = new ArrayList<>(nums);` creation d'une liste contenant n éléments donc $O(n)$
- Les variables `ai`, `bi`, `sum` sont constantes donc $O(1+1+1)$
- La complexité spatiale est donc de $O(3+n) = O(n)$

2. Décrivez la logique de votre solution optimale en espace et analysez la complexité temporelle et spatiale.

Logique de la solution:

- La solution vise à trouver une paire d'éléments dans une liste `nums` dont la somme est égale à `k`. Il y a deux boucles (externe et interne), la boucle externe parcourt chaque élément `a = nums.get(i)`, la boucle interne parcourt tous les éléments suivants `b = nums.get(j)` (avec `j > i`) pour éviter de tester deux fois la même paire. Si `a + b == k`, on retourne directement la paire `(a, b)`. Si aucune paire ne satisfait la condition, la fonction retourne `null`.

Complexité temporelle

- L'initialisation de `a` et `b` se fait en $O(1)$
- La boucle externe s'exécute n fois
 - `a = nums.get(a)` se fait en $O(1)$
 - Pour chaque `i`, la boucle interne s'exécute environ $n - i - 1$ fois, soit $(n-1) + (n-2) + \dots + 1$ ce qui revient à l'équation $\sigma \sim n(n+1)/2$, donc la boucle au complet est $O(n^2)$
- La fonction s'exécute en $O(n^2)$

Complexité spatiale

- Les variables `a`, `b`, `i`, `j` sont constantes donc $O(1+1+1+1)$
- La complexité spatiale est donc de $O(4) = O(1)$

3. Pouvez-vous être certains que vos solutions optimales le sont vraiment? justifiez.

Solution optimale en temps:

Pour une liste non triée, il est nécessaire de parcourir tous les éléments au moins une fois pour trouver la paire, ce qui impose $O(n)$ minimum. Ajouter un tri permet ensuite d'utiliser la technique des deux index (`ai` et `bi`) pour éviter de tester toutes les paires, ce qui évite la complexité $O(n^2)$ d'une double boucle. Une alternative serait d'utiliser un `HashSet` pour stocker les valeurs déjà vues et vérifier si `k - a` existe ce qui donnerait $O(n)$ en temps, mais la complexité spatiale resterait quand même à $O(n)$.

Solution optimale en espace:

La solution est optimale en espace, car aucune structure supplémentaire n'est utilisée, et ce n'est pas possible de réduire $O(1)$. La double boucle visite toutes les paires possibles, ce qui est nécessaire si l'on ne veut pas augmenter la complexité spatiale.

4. Décrivez brièvement votre utilisation des SIAG dans ce TP (questions et réponses obtenus, etc.).

SIAG utilisée pour:

- Vérification des complexités temporelle et spatiale du code
- Justification des notions d'optimalité en temps et en espace
- Corrections des fautes d'orthographe

Conclusions et observations tirées de l'utilisation des SIAG:

ChatGPT a permis de clarifier certaines incompréhensions sur la matière de ce tp et a permis de vérifier la complexité spatiale et temporelle du code.